

## PART A

## 20 Questions

Q1. What happens in the Binary Search algorithm if the element being searched is not found:

- ☒ A. The index with a default value which is -1 will be returned indicates the element that being searched was not in the list.
- ☐ B. The index with the last index value will be returned indicates the element that is being searched was not in the list.
- ☐ C. The middle index value will be returned to indicate the element that is being searched was not in the list.
- ☐ D. The first index value will be returned to indicate the element that is being searched was not in the list.

Q2. Which of the following is a disadvantage of linear search?

- ☐ A. Requires more space compared to other searching algorithms.
- ☒ B. Greater time complexities compared to other searching algorithms
- ☐ C. Not easy to understand.
- ☐ D. Dynamic size.

```
4 int search(int array[], int n, int x) {
5
6     int i=0;
7     int index=-1;
8     int count = 0;
9
10    while(i<n) {
11        if (array[i] == x)
12            index=i;
13        i++;
14        count++;
15    }
16
17    cout<<"Loop in "<<count<<" times "<<endl;
18    return index;
19 }
20
21 int main() {
22     int array[] = {2, 11, 0, 11, 9, 111, 21};
23     int x = 11;
24     int n = 7;
25
26     int result = search(array, n, x);
27
28     (result == -1) ? cout << "Element not found" : cout << "Element found at index: " << result;
29 }
```

Q3- Given program 1 above. Identify the **CORRECT** output of Program 1:

|   |                                              |                                    |                                              |
|---|----------------------------------------------|------------------------------------|----------------------------------------------|
| A | Loop in 7 times<br>Element found at index: 3 | <input checked="" type="radio"/> C | Loop in 1 times<br>Element found at index: 1 |
| B | Loop in 7 times<br>Element found at index: 5 | <input type="radio"/> D            | Loop in 4 times<br>Element found at index: 3 |

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

Q4. If the list cannot be found at the index middle, then the Binary searching time is,

- A.  $O(1)$
- B.  $O(n)$
- C.  $O(2^n)$
- ☒ D.  $O(\log_2 n)$

Q5.

What will be the value of "sum" after the following code snippet terminates?

```
void count(ListNode* nod) {
    /*
    The LinkedList is defined as:
    head point to the first node in the List
    nod-> data = value of the node
    nod-> next = address of next element from the node
    The List is 2 -> 4 -> 6 -> 8 -> 10
    */
    nod=head;
    nod=nod->next;
    int sum = 0;
    while (nod != NULL) {
        sum += nod -> data;
        nod = nod -> next;
    }
    cout << sum << endl;
}
```

*we already declare here that current node is the next node of the current node(2).*

*current node is not 2, but 4. why?*

- A. 10
- ☒ B. 28
- C. 30
- D. 18

Q6. Which type of linked list stores the address of the head node in the next pointer of the last node?

- A. Singly link list
- B. Doubly link list
- ☒ C. Circular link list
- D. Array linear list

Q7. Which of the following statements is **FALSE**?

- A. Arrays have better cache locality than a linked list
- B. Random access to elements of the linked list is not possible ✓
- C. The size of the linked list is dynamic and can be changed as needed. ✓
- ☒ D. Link lists need a checking mechanism during insertion to avoid overflow problems.

Q8. Which of the following linked list operations takes  $O(1)$  time?

- A. Find the number of nodes in a linked list
- B. Find certain elements in the linked list
- C. Insert element at the second last of the linked list
- D. Insert the element at the front of the linked list**

Q9.

Choose the correct final output for the following sequence of stack operations.  
(**Bold** is the top element in the stack.)

push (m)  
push (n)  
pop  
push (q)  
push (t)  
pop  
push (k)

- A. m q k**
- B. m n q t k
- C. k q m
- D. m k

Q10. Which one of the following is **FALSE** about the stack?

- i. It could be implemented using arrays. ✓
- ii. It follows LIFO ✓
- iii. The top of the stack contains the first inserted element.** → linked list
- iv. Elements are stored in a distributed manner. →

A. ~~i & iii~~

B. ~~ii & iii~~

**C. ii & iv**

D. ~~ii & iv~~

Q11.

Given postfix expression is : **20 5 + 32 8 / - 3 +**

What is the value after evaluation of the above expression?

- A. 24**
- B. 32
- C. 21
- 33

Q12

Convert infix expression:  $p - (b * c) + d/e$  to prefix expression. Answer is :

- ☐ A.  $-p*bc/+de$                       C.  $p-bc*de/+$   
☒ B.  $+ -p*bc/de$                       D.  $-pbc*+/de$

Q13 A linear, if implemented using an array of size MAX\_SIZE, gets full when?

- ☒ A.  $back = MAX\_SIZE - 1$   
B.  $front = (back + 1) \bmod MAX\_SIZE$   
C.  $front = back + 1$   
D.  $back = front$

Q14 Which of the following is true about linked list implementation of queue ?

i. In `enqueue isFull()` method no need to be called first before insert at the back of queue ✓

ii. In `dequeue isEmpty()` method should be called first before delete at the back of the queue if the queue is empty, what is there to delete?

iii. In `enqueue` operation, new nodes are inserted at the beginning of linked list, then in `dequeue` operation, nodes removed from end. ✗

iv. In `enqueue` operation, new nodes are inserted at the end of linked list, in `dequeue` operation, nodes removed from front. ✓

- ~~A. i, ii, iii~~    ~~B. ii and iv~~    ☒ C. i and iv    D. Only iv

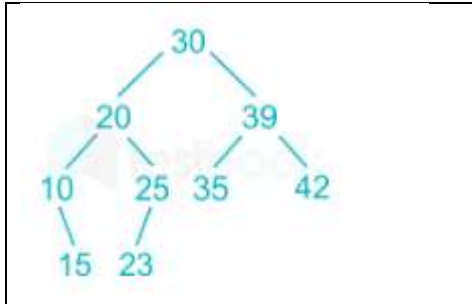
Q15 In a circular queue implementation using array of size 5, the array index starts with 0 where front and rear values are 2 and 3 respectively. Determine the array index at which the insertion of the next element will take place.

- ☒ A. 4    use  $front = (rear + 1) \% \text{max\_array}$   
B. 5  
C. 1  
D. 0

Q16 Which of the following statements is FALSE about **linear array implementation** of queue ?

- A. Enqueue operation is not allowed to the empty spaces at the front of the array ✓
- B. Enqueue operation is not allowed if the rear index is at the end of the array ✓
- ☒ C. The subsequent index of rear and front is determined using arithmetic (%) ✓
- D. Only rear is used to determine whether the queue is full ✓

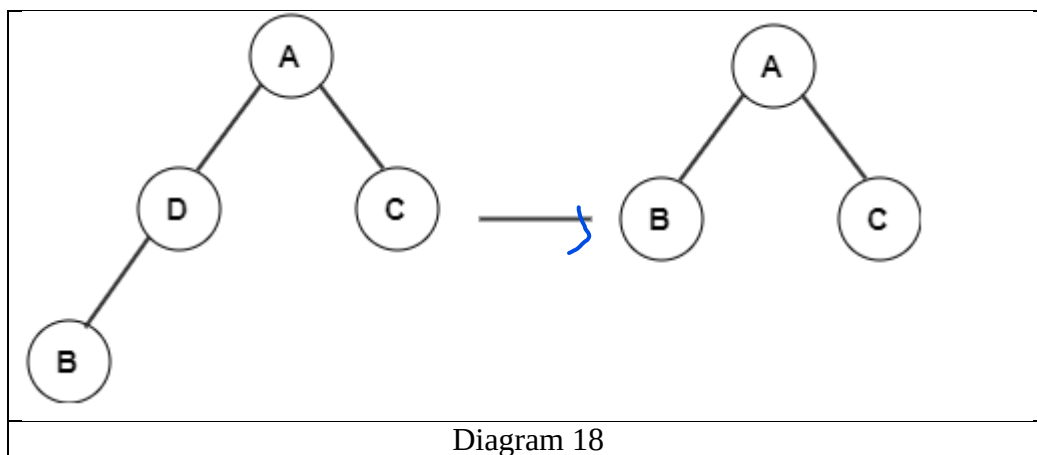
Q17 Given BST above :



In-order traversal of the tree above is :

- ☒ A. 15 10 23 25 20 35 42 39 30 ✓
- B. 10 15 20 23 25 30 35 39 42
- C. 30 20 10 15 25 23 39 35 42
- D. 30 20 10 25 15 23 35 39 42

Q18 What operation does the following diagram 18 depict?



- A) inserting a leaf node ✗
  - B) inserting an internal node ✗
  - ☒ C) deleting a node with 0 or 1 child ✓
- Node B is already there

D) deleting a node with 2 children ✗

Q19 Which of the following properties are obeyed by all three tree – traversals?

A) Left subtrees are visited before right subtrees ✓

B) Right subtrees are visited before left subtrees ✗

C) Root node is visited before left subtree

D) Root node is visited before right subtree

Q20 The function print() receives root of a Binary Search Tree (BST) and a positive integer k as arguments.

```
// A BST node
struct node {
    int data;
    struct node *left, *right;
};

int count = 0;

void print(struct node *root, int k)
{
    if (root != NULL && count <= k)
    {
        print(root->right, k);
        count++;
        if (count == k)
            printf("%d ", root->data);
        print(root->left, k);
    }
}
```

A. Prints the  $k$ th smallest element in BST ✗

B. Prints the  $k$ th largest element in BST

C. Prints the leftmost node at level  $k$  from root ✗

D. Prints the rightmost node at level  $k$  from root

## PART A

THIS PART CONSISTS OF THREE (3) QUESTIONS. ANSWER ALL QUESTIONS IN THIS PART.

### Question 1

[12 marks]

Given **DATA** array of integers as in Figure 1, answer question a) and b).

| index | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] |
|-------|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|       | 250 | 270 | 111 | 135 | 98  | 100 | 125 | 170 | 190 |

Figure 1: **DATA** array

- a) Name the searching technique suitable to be performed on the data array in Figure 1. Give the reason to justify your answer.

[ 2 marks ]

- b) What should be done to the **DATA** array in Figure 1 to reduce the complexity time of the searching process? Explain your answer.

[ 2 marks ]

Given **NUM** array of integers as in Figure 2, answer question c), d) and e).

| index | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] |
|-------|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|       | 1   | 3   | 5   | 7   | 9   | 11  | 13  | 17  | 19  |

Figure 2: **NUM** array

- c) Perform binary search for searching **key 20** on the **NUM** array in Figure 2. Show the tracing of your search using variables **left**, **right**, **middle**, **NUM[middle]** and **found** as shown in the table format below.

[ 3 marks ]

| left | right | middle | NUM[middle] | found |
|------|-------|--------|-------------|-------|
| 0    | 8     | 4      | 9           | false |
| 5    | 8     | 6      | 13          | false |
| 7    | 8     | 7      | 17          | false |
| 8    | 8     | 8      | 19          | false |

- d) Perform binary search for searching **key 5** on the **NUM** array in Figure 2. Show the tracing of your search using variables **left**, **right**, **middle**, **NUM[middle]** and **found** as shown in the table format below.

[ 3 marks ]

| left | right | middle | NUM[middle] | found |
|------|-------|--------|-------------|-------|
|      |       |        |             |       |
|      |       |        |             |       |
|      |       |        |             |       |

- e) Discuss the differences between the number of steps and the complexity time when searching for **key equals to 19** on **NUM** array in Figure 2 using binary search and sequential search technique.

[ 2 marks ]

## Question 2

**[19 marks]**

- a) A queue can be implemented using circular array or circular linked list data structure. Give one advantage of queue using circular array and one advantage of queue using circular linked list. What kind of problem that both implementation can solve.

[2 marks]

- b) Give the differences between the implementation of the queue implementation array and queue implementation circular array in the following operations:

- i) Queue declaration
- ii) **isEmpty()** function.
- iii) **isFull()** function.

[3 marks]



c) Figure 3 shows the current content of a circular queue.

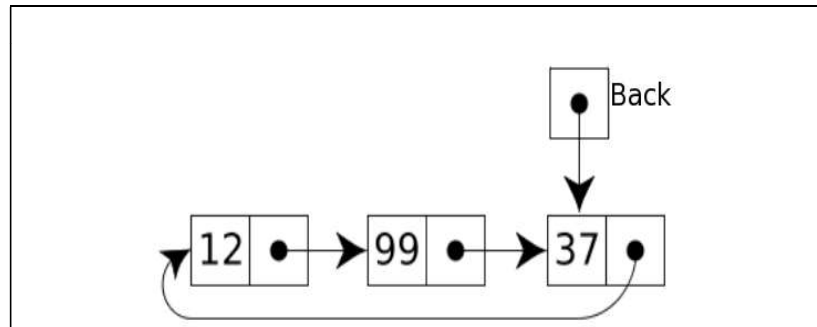


Figure 3: The current queue content

Redraw the queue in Figure 4 that shows the current content of the queue after the sequence of instructions in Figure 4 is executed.

[3 marks]

```
Queue -> enQ(9) ;
Queue -> enQ(4) ;
Queue -> deQ() ;
```

Figure 4 : Queue instructions

Answer the following questions, which deal with operations on queue implementation array.

Figure 5 shows Queue 1 with maximum content of 4 items. Currently, there are two items already in the queue after **enQueue (A)** and **enQueue (B)** operations have been executed.

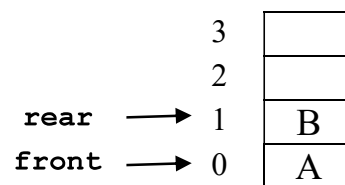


Figure 5: Queue 1

d) Perform the following tasks to Queue 1:

- i. Redraw Queue 1 after **enQueue (G)** followed by **enQueue (F)** . Label the correct location of the **front** and **rear**.
- ii. Redraw the queue in question (d- i) after **deQueue ()** and **enQueue (G)** . Label the correct location of the **front** and **rear**.

[3 marks]

- e) Figure 6, shows a circular queue called Queue 2, which fits maximum of 8 items. The current content of Queue 2 are as shown in the figure.

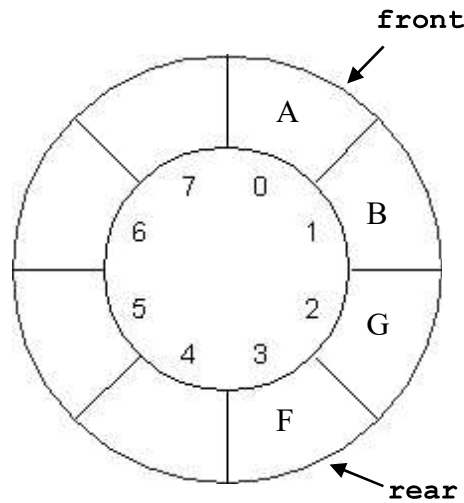


Figure 6: Queue 2

- i. Redraw Queue 2 after **E, J, M, H** are **enQueue** in sequence. Label the correct location of the **front** and **rear**. [2 marks]
- ii. Redraw updated queue in question (i) after **3 deQueue** operations are performed . Label the correct location of the **front** and **rear**. [2 marks]
- iii. List all the deQueue items in question (ii) in FIFO sequence. [1.5 marks]
- iv. Redraw updated queue in (ii) after **R and Q** are **enQueue** in sequence. Label the correct location of the **front** and **rear**. [1 marks]
- v. After a series of **enQueue** and **deQueue** operations, compare the efficiency of memory usage for Queue1 and Queue2. [1.5 marks]

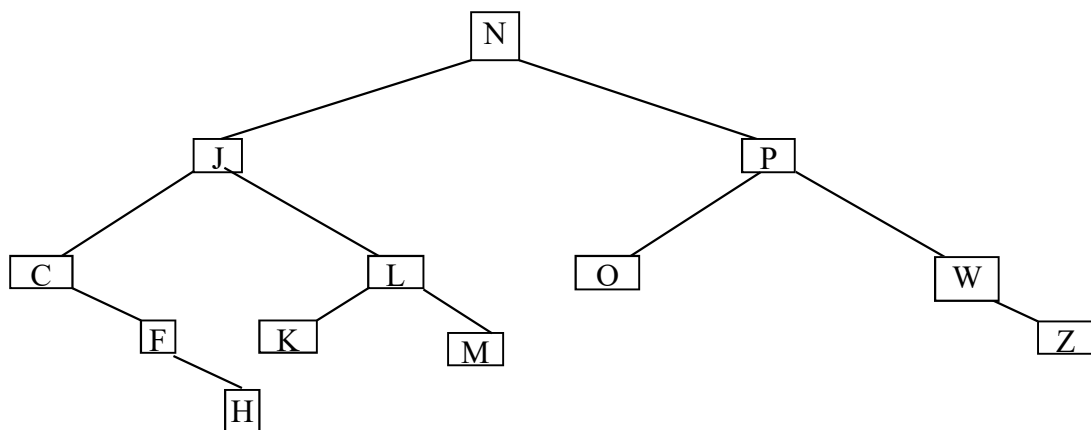
**Question 3**

**[19 marks]**

- a) Discuss the advantage of full tree and complete tree in terms of complexity related to insertion, deletion and searching operations. [2 marks]
- b) Discuss the drawbacks of Binary Search Tree in terms of complexity related to insertion, deletion and searching operations. [2 marks]
- c) Draw the expression tree for the following arithmetic expression. [3 marks]

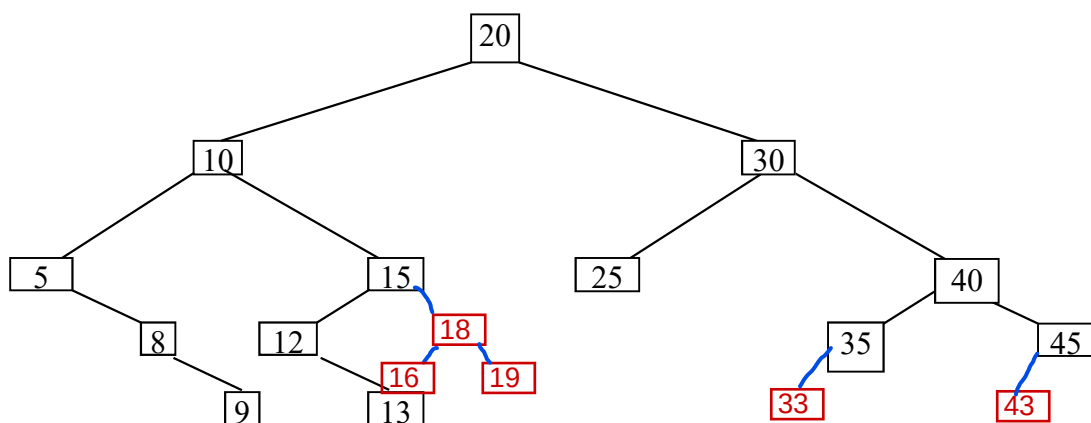
$$a * b - (c * 11 / d) + (e + f * 5)$$

- d) Give the inorder, preorder and postorder traversal of the tree in Figure 7. [6 marks]



**Figure 7: Binary Search Tree of char value**

Given the following binary search tree in Figure 8, answer question 5 and 6.



**Figure 8: Binary Search Tree of int value**

e) Redraw the tree in Figure 8 after nodes with values **18, 16, 19, 33 and 43** are inserted in sequence. [2 marks]

f) Redraw the tree in Figure 8 after the following nodes are deleted **in sequence**. Show the new tree after every deletion.

i) 25

ii) 20

iii) 10

[4 marks]

**PART B**

**THIS PART CONSISTS OF 2 QUESTIONS. SELECT AND ANSWER ONLY ONE (1) QUESTION IN THIS PART.**

**Question 1****[20 marks]**

You are given the following code as shown in Figure 9. When the code execute, the result can be represented in Figure 10.

```

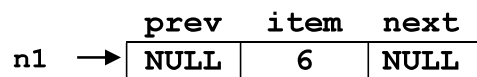
struct NODE {
    int item;
    NODE* prev;
    NODE* next;
};

// main function
int main(int argc, char *argv[]) {
    NODE* n1 = new NODE;
    n1->item = 6;
    n1->prev = NULL;
    n1->next = NULL;

    //assume the additional code segment for question a - d is here

} //end main

```

**Figure 9 : Linked List Codes****Figure 10: Current execution state with node n1**

- a) Show in a diagram, how nodes **n1**, **n2** and **n3** are interconnected to each other, when the following code segment is executed.

**[3 marks]**

```

NODE* n2 = new NODE;
n2->item = 3;
n2->prev = n1;
n2->next = NULL;

NODE* n3 = new NODE;
n3->item = 8;
n3->prev = n2;
n3->next = NULL;

```

- b) Show in a diagram, how nodes **n1**, **n2** and **n3** are interconnected to each other, when the following code segment is executed. [3 marks]

```
n1->next = n2;
n2->next = n3;
```

- c) A new node, **n4** is created as follows:

```
NODE* n4 = new NODE;
n4->item = 7;
n4->prev = NULL;
n4->next = NULL;
```

Show in a diagram, how nodes **n1**, **n2**, **n3** and **n4** are interconnected to each other, when the code is executed

[3 marks]

- d) Write a code segment to delete node **n2** and replace it with the node **n4** [3 marks]

- e) Another code segment is added as follows:

```
NODE * print_ptr;
print_ptr = n1;
```

Write a code segment to display the value of item in all the nodes, using **print\_ptr** [3 marks]

- f) Bill tracker application called “Bills Reminder” need to be developed. The Bills Reminder has a basic functions that store types of bill payment, bill payment amount and the monthly due date. If linked list is used as a data structure to be used in designing the application, describe how linked list can be used as the data structure that able to store the information of the bill every month as shown in Table 1.

**Table 1:** Sample of bills, the amount, and due date for one month

| Bill        | Amount | Due date for every month |
|-------------|--------|--------------------------|
| Celcom      | 40     | 1                        |
| Electricity | 30     | 1                        |
| Credit card | 900    | 16                       |
| House rent  | 200    | 29                       |

[5 marks]

**Question 2**

**[20 marks]**

- a) Consider the following source codes in Program Segment 1 that will be implemented on stack using array. The **size of the stack is 5**.

|    |                                                  |
|----|--------------------------------------------------|
| 1  | <b>Stack s;</b>                                  |
| 2  | <b>string a1, a2, a3;</b>                        |
| 3  | <b>:</b>                                         |
| 4  | <b>:</b>                                         |
| 5  | <b>a1=s.stackTop( );</b>                         |
| 6  | <b>a2= "Luck";</b>                               |
| 7  | <b>s.push(a2);</b>                               |
| 8  | <b>s.push(a1+a2);</b>                            |
| 9  | <b>s.push("Congratulations");</b>                |
| 10 | <b>a2=s.stackTop( );</b>                         |
| 11 | <b>s.pop();</b>                                  |
| 12 | <b>s.pop();</b>                                  |
| 13 | <b>s.push("Bye!");</b>                           |
| 14 | <b>a3=s.stackTop( );</b>                         |
| 15 | <b>s.pop();</b>                                  |
| 16 | <b>cout&lt;&lt; a1 &lt;&lt; a3 &lt;&lt;endl;</b> |
| 17 | <b>cout&lt;&lt; a2 &lt;&lt;endl;</b>             |

**Program Segment 1:** Stack source codes

- i) Show the content of the stack after each implementation of the sources codes based on the situation below. Assume the current content of the stack before statement Line 5 is executed as shown in first stack in the Table 2. [3 marks]

Element  
of array

Content of Stack

|     |                                        |                                              |                            |                             |                             |                             |
|-----|----------------------------------------|----------------------------------------------|----------------------------|-----------------------------|-----------------------------|-----------------------------|
| [4] |                                        |                                              |                            |                             |                             |                             |
| [3] |                                        |                                              |                            |                             |                             |                             |
| [2] | <b>Good</b>                            |                                              |                            |                             |                             |                             |
| [1] | <b>Bye!</b>                            |                                              |                            |                             |                             |                             |
| [0] | <b>Thank<br/>You</b>                   |                                              |                            |                             |                             |                             |
|     | Current<br>content<br>before<br>Line 5 | Content<br>After Line<br>5-8 are<br>executed | Content<br>After Line<br>9 | Content<br>After Line<br>11 | Content<br>After Line<br>12 | Content<br>After Line<br>13 |
|     |                                        |                                              |                            |                             | Content<br>After Line<br>15 |                             |

**Table 2**

ii) What will be displayed by the program in Program Segment 1? [2 mark]

b) Using stack operations, show step-by-step conversion of the following infix expression to postfix notation. Use the format given in the table below.

$$A - B * (C + D * E) / F$$

[5 marks]

| Infix | Stack | Postfix |
|-------|-------|---------|
|       |       |         |
|       |       |         |

c) Evaluate the following postfix expression using stack concept: **3 6 – 10 + 5 \***  
 You need to use the format given in the table below in order to show the evaluation process. [4 marks]

| Postfix | Ch | Opr | Opn1 | Opn2 | Result | Stack |
|---------|----|-----|------|------|--------|-------|
|         |    |     |      |      |        |       |
|         |    |     |      |      |        |       |
|         |    |     |      |      |        |       |

d) A compiler need to keep track of a return address of next statement to be executed when return from a function call as well as all parameters passed to the calling function. An example of a program (partial codes) is given in Figure 11, in which a main function calls function A. Function A calls function B, which then calls function C. Bellow are steps for the complier when executing the program:

|      |      |                                                                 |
|------|------|-----------------------------------------------------------------|
| step | i.   | the main function calls function A                              |
| step | ii.  | function A calls function B                                     |
| step | iii. | function B calls function C                                     |
| step | iv.  | completion of executing function C and returns to function B    |
| step | v.   | completion of executing function B and returns to function A    |
| step | vi.  | completion of executing function A and returns to main function |



| address | partial code                 |
|---------|------------------------------|
| 1000    | int main() {                 |
| 1004    | int a, b, c;                 |
| ....    | .....                        |
| 1050    | function A(a,b,c);           |
| 1054    | cout << a << b << c << endl; |
| ....    | .....                        |
| 1100    | }                            |

| address | partial code                           |
|---------|----------------------------------------|
| 200     | function A(int a, int b, int c) {      |
| 204     | int d, e;                              |
| ....    | .....                                  |
| 300     | function B(a,b,d,e);                   |
| 304     | cout << a << b << c << d << e << endl; |
| ....    | .....                                  |
| 500     | }                                      |

| address | partial code                                |
|---------|---------------------------------------------|
| 100     | function B(int a, int b, int c, int d) {    |
| 104     | int e, f;                                   |
| ....    | .....                                       |
| 150     | function C(a,d,e,f);                        |
| 154     | cout << a << b << c << d << e << f << endl; |
| ....    | .....                                       |
| 198     | }                                           |

| address | partial code                                |
|---------|---------------------------------------------|
| 800     | function C(int a, int b, int c, int d) {    |
| 804     | int g, h;                                   |
| ....    | .....                                       |
| 900     | cout << a << b << c << d << g << h << endl; |
| 904     | .....                                       |
| ....    | .....                                       |
| 998     | cout << "end of nested call" << endl;       |
| ....    | .....                                       |
| 998     | }                                           |

Figure 11: Partial codes for function call

- i) What is the most suitable data structure that the compiler should use in keeping track of returned addresses and parameters when executing the program. [1 mark]
- ii) Draw a diagram to show the content of the data structure for each step of execution (steps i – iv) in illustrating the process. Based on the diagram, provide description of the operation on the data structure. [5 mark]